



Universidade do Minho
Escola de Engenharia
Mestrado Engenharia Informática

Unidade curricular de Aprendizagem Profunda

Ano académico 2025/2026

Relatório final do trabalho do módulo 1

Diogo Rodrigues(PG60244) Gabriel Antunes(PG60259)
Vicente Martins(PG58812) Simão Alves(PG58810) Rui Gonçalves(PG59847)

April 6, 2026

AP

Índice

1	Introdução	1
2	Construção dos Datasets	1
2.1	Método 1: Abordagem Temática e Dataset de Teste	1
2.2	Método 2: Expansão e Controlo Temático	2
3	Modelo Numpy	3
4	Modelo Pytorch	5
5	Discussão dos Resultados	6
5.1	Numpy	6
5.2	Pytorch	8
6	Conclusão	11

List of Figures

1	Comparação dos graficos de Loss e Accuracy de diferentes modelos. . .	6
2	Matrizes de confusão dos resultados da previsão do dataset de teste. . .	8
3	Gráficos de treino do transformer	9
4	Matriz de confusão	10

List of Tables

1	Comparação de desempenho dos modelos	7
2	Comparação de desempenho dos modelos em pytorch	9
3	Relatório de classificação	10

1 Introdução

A detecção de texto gerado por Inteligência Artificial exige uma abordagem que combine técnicas clássicas de Processamento de Linguagem Natural com arquiteturas modernas de redes neurais. Neste projeto, procurámos resolver este problema através de duas vertentes metodológicas distintas. Primeiro, a implementação de modelos de raiz utilizando exclusivamente a biblioteca Numpy, garantindo a compreensão profunda de algoritmos como Regressão Logística e Deep Neural Networks (DNNs). Além desta, a exploração de modelos complexos em PyTorch, incluindo (LSTMs/GRUs) e Transformers. O sistema é treinado e validado com o propósito de classificar pequenos textos em cinco categorias distintas, sendo estas Human, Google, Anthropic, OpenAI e Meta, o que permite não só detetar se o texto foi escrito por inteligência artificial, mas atribuir a autoria ao modelo ou empresa responsável pela sua geração.

2 Construção dos Datasets

A qualidade de um modelo de classificação de texto está intrinsecamente ligada à qualidade, representatividade e volume dos dados de treino. Para isso, o desenvolvimento do dataset final resultou de um processo iterativo composto por duas abordagens metodológicas, onde se deu sempre prioridade ao tema dos textos (ciências naturais) e ao tamanho dos mesmos, já que os textos a classificar teriam sempre entre 100 a 120 palavras.

2.1 Método 1: Abordagem Temática e Dataset de Teste

A primeira abordagem exploratória teve como base um dataset extraído do Kaggle intitulado 'GPT vs. Human: A Corpus of Research Abstracts', que consiste num compilado de resumos (abstracts) de estudos científicos publicados. Deste corpus, foram selecionados 33 temas distintos. Com o intuito de construir as classes sintéticas, cada um dos quatro LLMs (OpenAI, Meta, Google e Anthropic) foram instruídos a gerar 50 textos sobre cada um desses 33 tópicos.

No entanto, a análise exploratória deste primeiro conjunto de dados revelou limitações críticas. Conforme alertado pelo docente da disciplina, a distribuição das classes encontrava-se severamente desbalanceada. A classe representativa de textos "Humanos" continha apenas cerca de 900 amostras (representando essencialmente 1/3 do dataset), contrastando com as aproximadamente 1.500 amostras presentes em cada uma das classes geradas por IA.

Para tentar mitigar este problema, implementaram-se várias estratégias corretivas como a penalização por pesos (Class Weights) e foram criadas múltiplas versões de amostragem

com undersampling para nivelar todas as classes pelo mesmo número exato de casos e outra com a proporção original com a classe humana a representar 1/3 dos dados.

Apesar destes esforços iterativos, a abordagem esbarrou na escassez de dados. Independentemente da técnica de balanceamento aplicada, o volume total de textos revelou-se demasiado baixo para o treino robusto das arquiteturas de Deep Learning propostas, resultando inevitavelmente em problemas de overfitting.

Perante estas limitações, concluiu-se que este conjunto de dados não era adequado para o treino primário. Em vez de ser descartado, foi estrategicamente isolado para servir como um conjunto de teste cego independente (referenciado no código como `dataset_final_v3.csv`), garantindo assim uma avaliação final do modelo num conjunto de dados estruturalmente distinto dos dados de treino.

2.2 Método 2: Expansão e Controlo Temático

Para resolver a escassez de dados e garantir que os modelos aprendiam características intrínsecas (estilo) em vez de se viciarem no tópico, desenvolveu-se uma segunda abordagem metodológica.

O processo de geração consistiu em extrair as primeiras 10 palavras de milhares de parágrafos humanos originais da Wikipédia. Em seguida, essas 10 palavras foram enviadas através de chamadas à API para os vários LLMs com um Prompt rigoroso, solicitando que a inteligência artificial completasse o resto do parágrafo. Forçou-se adicionalmente uma restrição de tamanho, obrigando as respostas a conterem entre 80 a 120 palavras e textos gerados fora desta margem foram automaticamente rejeitados pelo script, garantindo a consistência do comprimento das sequências.

A extração de mais de 40.000 textos gerados por IA exigiu uma arquitetura paralela (Multithreading) para otimizar o tempo de geração, lidando simultaneamente com mecanismos de Rate Limiting (Erros 429) e orçamentos limitados por níveis de acesso nas plataformas comerciais.

Devido a quotas de utilização estritas na API da Anthropic, a geração para esta classe foi interrompida defensivamente de forma prematura. A distribuição consolidada no "Dataset Mãe" ficou assim constituída:

- Humanos (Wikipédia): 10.900 amostras
- Meta (Llama via Groq): 10.900 amostras
- OpenAI: 10.900 amostras
- Google (Gemini): 10.900 amostras

- Anthropic (Claude): 7.200 amostras

Este ligeiro desbalanceamento da classe Anthropic provou ser irrelevante para a convergência das redes, existindo um volume de exemplos perfeitamente adequado para a extração de features latentes.

A decisão de utilizar as mesmas 10 palavras iniciais para as cinco variantes (1 Humano + 4 IAs) no Método 2 introduziu um desafio crítico de Data Leakage, pois, se a divisão dos dados entre os conjuntos de Treino e Validação fosse aleatória, diferentes versões de um mesmo fragmento poderiam acabar em conjuntos distintos, permitindo ao modelo memorizar o contexto e inflacionar artificialmente as métricas.

Para mitigar este risco, implementou-se um particionamento agrupado (Group Split). Utilizando a chave original de cada fragmento (coluna Original_ID) em conjunto com a função GroupShuffleSplit, garantiu-se que as cinco variações derivadas de um mesmo excerto transitavam sempre em bloco para o mesmo subset.

Após a geração, todos os ficheiros foram fundidos num único conjunto, limpavam-se eventuais valores nulos resultantes de falhas de rede e aplicou-se um shuffle aleatório. O particionamento final foi realizado isolando 80% dos dados agrupados para Treino e 20% para Validação (Early Stopping). Como delineado na secção 2.1, o teste final de robustez seria realizado com o dataset inicialmente gerado, garantindo um ciclo de vida de dados rigoroso, imaculado e representativo dos desafios reais.

3 Modelo Numpy

Para o nosso primeiro modelo, tal como foi proposto na tarefa 2 do projeto, mantivemos durante todas as submissões um modelo com base em código próprio utilizando apenas numpy. Para atingir o modelo que decidimos manter na submissão final, efetuámos diversos processos para garantir a maior *accuracy* possível nos datasets disponibilizados pelo professor.

A primeira etapa do processo foi fazer uma pequena limpeza e normalização nos dados, de modo a uniformizar os diferentes padrões e permitir ao modelo estabelecer correspondências entre as origens dos diferentes textos. Para isso utilizamos algumas funções pré-definidas em python como `.lower()` e também ao módulo de expressões regulares, para remover alguns padrões recorrentes.

Depois de normalizados conseguimos partir de um estado em que os dados se encontram nas mesmas condições. Para exponenciar a qualidade de previsão do nosso modelo passámos para a etapa do Feature Engineering. Nesta recorremos a três mecanismos principais:

- **Bag of Words:** onde um texto é transformado numa representação numérica em

que cada posição de um vetor regista o número de ocorrências de uma determinada palavra. Esta estratégia ignora a ordem e o contexto das palavras, focando-se apenas na frequência.

- **TF-IDF**: medida estatística que avalia a importância de uma palavra num texto. Tem a vantagem de destacar termos mais relevantes, penalizando palavras muito comuns, como "o", "as" e "de", que normalmente não ajudam a distinguir classes.
- **Bi-grams**: ao criar novas features que não existiam originalmente, os bi-grams dão ao modelo alguma noção de contexto ao capturar pares de palavras frequentemente usados em conjunto, e não apenas palavras isoladas.

O modelo foi implementado como uma rede neuronal totalmente conectada, construída de forma modular. Foram exploradas duas configurações principais:

- **Logistic Regression**: Implementada como uma única camada densa seguida de uma função de ativação Softmax. Serve como baseline simples e eficiente para classificação multi-classe.
- **Deep Neural Network (DNN)**: Composta por múltiplas camadas Dense Layers intercaladas com funções de ativação ReLU, permitindo capturar relações mais complexas entre as features.

A camada de saída utiliza a função **Softmax** produzindo uma distribuição de probabilidades sobre as classes. Para a função de perda foi utilizada a **Categorical Cross-Entropy**

O treino foi realizado através de mini-batch gradient descent com momentum, permitindo atualizações mais estáveis e rápidas dos pesos. Adicionalmente, foram incorporadas técnicas para melhorar a robustez do modelo:

- **Momentum**: Introduce memória nas atualizações dos gradientes, acelerando a convergência e reduzindo oscilações.
- **Early Stopping**: O treino é interrompido quando a performance no conjunto de validação deixa de melhorar durante um determinado número de épocas (patience), prevenindo overfitting.
- **Regularização L2**: Aplicada diretamente nos pesos durante o cálculo do gradiente, penalizando valores elevados e contribuindo para uma melhor generalização.

Foram realizados vários testes com diferentes combinações de features e arquiteturas. O modelo final foi selecionado com base no desempenho no conjunto de validação, tendo em conta métricas como accuracy e comportamento da função de perda ao longo das épocas.

4 Modelo Pytorch

Com a utilização da framework Pytorch foi feito uma pipeline para testar e comparar a eficácia de várias arquiteturas distintas.

Antes de alimentar as redes neuronais, o texto cru foi sujeito a um processo de normalização e conversão numérica, garantindo que todas as arquiteturas recebiam tensores consistentes, incluindo tokenização e limpeza, construção de vocabulário e encoding e padding.

Por forma a estabelecer termos de comparação sólidos, foram implementadas quatro arquiteturas com graus crescentes de complexidade:

- Simple (RNNClassifier): Modelo de base constituído por uma camada de Embedding, seguida de uma Recorrente Simple (RNN) e uma camada Linear de saída (alimentada pelo último hidden state). Com parâmetros básicos (embed_dim=200, hidden_dim=128, 1 camada), obteve uma exatidão de validação inicial de apenas 0.50, evidenciando dificuldade em reter contexto a longo prazo.
- LSTM (LSTMClassifier): Esta é uma evolução natural para resolver o problema do Vanishing Gradient, foi configurada para suportar bidirecionalidade (concatenando os estados forward e backward) e duas camadas ocultas, intercaladas com Dropout (0.5) para regularização.
- GRU (GRUClassifier): Operando com os mesmos princípios bidirecionais da LSTM, mas com menos parâmetros treináveis (apenas 1 camada e Dropout de 0.3). Mostrou-se altamente eficiente para a tarefa de classificação de texto.
- Transformer (TransformerClassifier): A arquitetura mais sofisticada do projeto, implementada inteiramente de raiz (sem recurso à layer nativa nn.TransformerEncoder do PyTorch). Esta implementação incluiu módulos de Positional Encoding (codificação sinusoidal), Self-Attention, Multi-Head Attention (com 4 cabeças), Feed-Forward Networks e Residual Connections com LayerNorm. Em vez de focar apenas no primeiro ou último token, adotou-se a técnica de Mean Pooling sobre a dimensão da sequência, permitindo que todas as posições contribuíssem para a previsão final.

O processo de aprendizagem decorreu sob o otimizador Adam (com learning rate de 0.001) e avaliado com a função de custo CrossEntropyLoss. Respeitaram-se as boas práticas do PyTorch, alternando rigorosamente entre os modos model.train() (ativando gradientes e dropout) e model.eval() juntamente com torch.no_grad() para avaliações.

Para a arquitetura Transformer, devido à sua suscetibilidade a instabilidades iniciais de treino, o loop foi robustecido com técnicas avançadas como Gradient Clipping (max_norm=1.0), de forma a prevenir a explosão de gradientes, e Early Stopping com checkpointing, para

que, em vez de treinar cegamente durante as 20 épocas estipuladas, o modelo parava o treino caso a Accuracy de validação não melhorasse durante 5 épocas consecutivas (patience=5). O sistema guarda continuamente o estado do melhor modelo (best_transformer.pt), restaurando-o automaticamente no final do processo para garantir que as previsões do teste final usavam os pesos ideais.

5 Discussão dos Resultados

5.1 Numpy

Nesta secção analisamos o desempenho dos modelos desenvolvidos em Numpy, comparando diferentes abordagens de feature engineering e arquiteturas. São apresentados os resultados de treino, validação e teste, bem como as respetivas matrizes de confusão, permitindo avaliar a capacidade de generalização dos modelos.

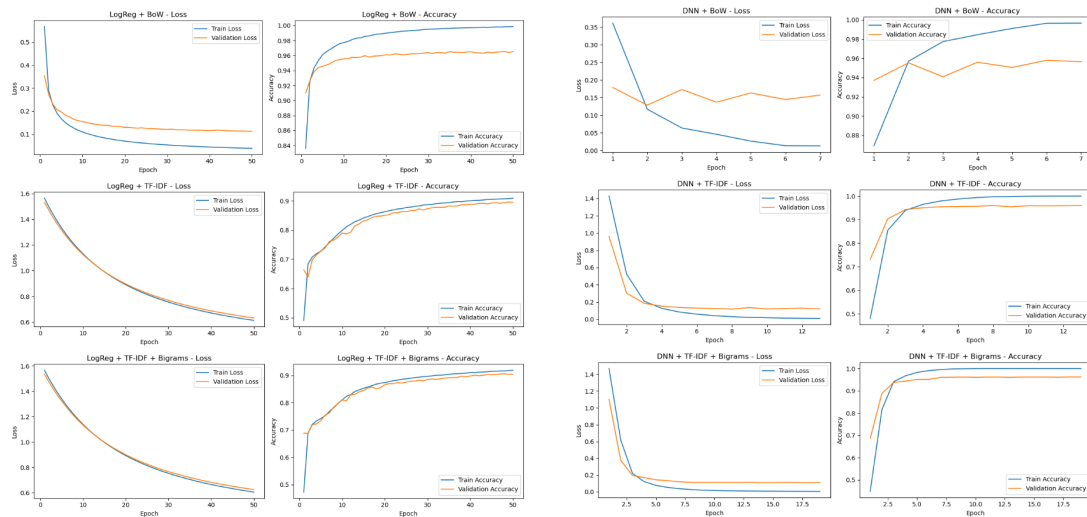


Figure 1: Comparação dos graficos de Loss e Accuracy de diferentes modelos.

A Figure 1 apresenta a evolução da função de loss e da accuracy ao longo das épocas para diferentes combinações de modelos e representações textuais.

De forma geral, observa-se que todos os modelos convergem rapidamente, com a loss a diminuir significativamente nas primeiras épocas e a estabilizar posteriormente. A accuracy de treino aproxima-se de valores elevados, enquanto a accuracy de validação estabiliza em valores ligeiramente inferiores.

Modelo	Validação	Teste
Logistic Regression (BoW)	0.965	0.968
Logistic Regression (TF-IDF)	0.895	0.900
Logistic Regression (TF-IDF + Bi-grams)	0.904	0.912
DNN (BoW)	0.955	0.963
DNN (TF-IDF)	0.960	0.962
DNN (TF-IDF + Bi-grams)	0.962	0.970

Table 1: Comparação de desempenho dos modelos

Na Table 1 verifica-se que todos os modelos atingem valores elevados de accuracy, superiores a 0.89 no conjunto de validação. O modelo Logistic Regression com Bag of Words destaca-se nesta fase, apresentando o melhor desempenho, com uma accuracy de aproximadamente 0.965. Este resultado evidencia que, mesmo com uma arquitetura simples, uma representação adequada dos dados pode ser suficiente para capturar padrões relevantes.

No entanto, ao analisar os resultados no conjunto de teste, observa-se uma ligeira alteração no desempenho relativo dos modelos. Embora a maioria mantenha valores elevados e próximos dos obtidos na validação, indicando boa capacidade de generalização, o modelo DNN com TF-IDF e bi-grams apresenta o melhor desempenho global, com uma accuracy de aproximadamente 0.970.

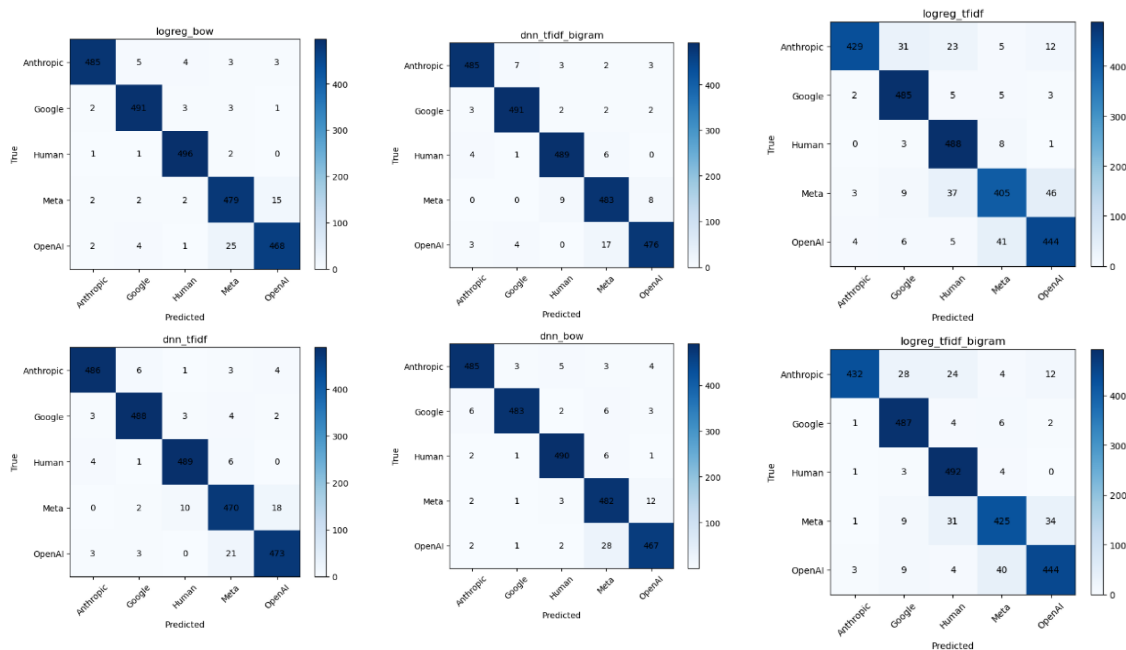


Figure 2: Matrizes de confusão dos resultados da previsão do dataset de teste.

A Figure 2 apresenta as matrizes de confusão para os diferentes modelos, permitindo uma análise mais detalhada dos erros de classificação.

De forma geral, verifica-se que a maioria das previsões se encontra na diagonal principal, indicando classificações corretas. No entanto, existem alguns erros entre classes semanticamente semelhantes, especialmente entre diferentes modelos de IA, o que sugere que estes partilham padrões linguísticos próximos.

5.2 Pytorch

Com vista a identificar a rede neuronal mais adequada para a tarefa de classificação de autoria (entre Humanos e os vários LLMs), implementaram-se quatro arquiteturas. Os resultados de validação e os respetivos hiperparâmetros principais encontram-se sumariados na tabela abaixo:

Esta progressão hierárquica de desempenho (RNN < LSTM < GRU < Transformer) corrobora a teoria fundamental de redes neurais para Processamento de Linguagem Natural (NLP) em que o Transformer provou ser amplamente superior devido aos mecanismo de Self-Attention. Dada esta superioridade, o Transformer foi a arquitetura seleccionada para as avaliações finais.

O processo de treino da arquitetura Transformer foi monitorizado ao longo de um limite máximo estipulado de 20 épocas. Para garantir estabilidade, aplicou-se Gradient Clipping (max_norm=1.0), e para prevenir o overfitting, configurou-se um mecanismo de Early

Modelo	Val. Accuracy (aprox.)	Hiperparâmetros Principais
RNN	50%	embed_dim=200, hidden=128, layers=1
LSTM	71%	embed_dim=200, hidden=128, layers=2, bidir=True, dropout=0.5
GRU	74%	embed_dim=200, hidden=128, layers=1, bidir=True, dropout=0.3
Transformer	93% (Interna)	embed_dim=128, heads=4, ff=256, layers=2, dropout=0.4

Table 2: Comparação de desempenho dos modelos em pytorch

Stopping com uma tolerância de 5 épocas.

O treino revelou um comportamento clássico de convergência:

- O melhor modelo foi guardado na época 8, atingindo uma Loss de Validação de 0.2144 e uma Accuracy de Validação de 93.15%.
- A partir da época 9, observou-se que a Loss de treino continuou a diminuir (chegando aos 0.1773 na época 13), enquanto a Loss de validação começou a estagnar e a oscilar, um indicador claro do início de overfitting.
- O Early Stopping cumpriu o seu propósito, interrompendo a execução na época 13 e restaurando automaticamente os pesos do checkpoint ótimo da época 8.

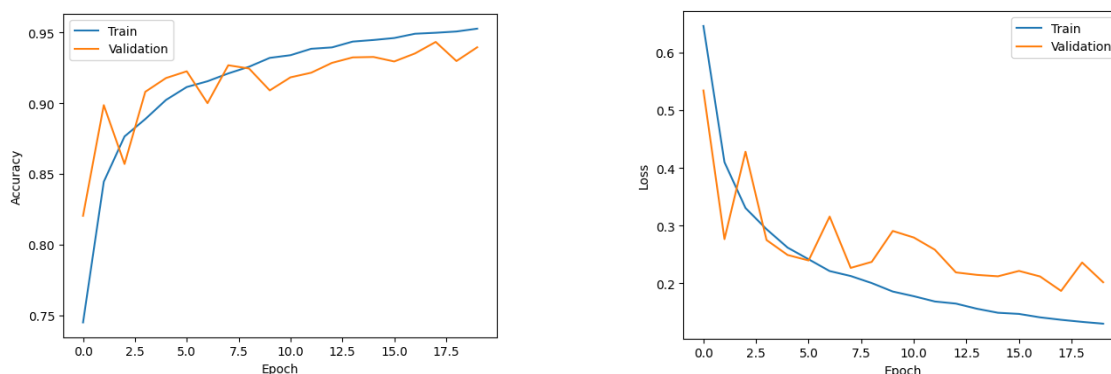


Figure 3: Gráficos de treino do transformer

Na avaliação final (Submissão 3) contra os 196 textos independentes do professor, o Transformer obteve uma Accuracy consolidada de 60.20% (118 previsões corretas). O relatório de classificação detalhado do pytorch expõe assimetrias notáveis entre as diferentes classes:

A classe Humana demonstrou ser a mais facilmente identificável pela rede neuronal,

Classe	Precision	Recall	F1-Score
Human	0.63	0.91	0.75
Anthropic	0.50	0.47	0.48
Google	0.69	0.51	0.59
OpenAI	0.33	0.14	0.20
Meta	0.64	0.62	0.63

Table 3: Relatório de classificação

alcançando um Recall expressivo de 91%. Isto sugere que o texto escrito por humanos possui imperfeições características, entropia e uma variância estilística que o distingue da cadência sintética gerada por máquina.

Em forte contraste, a classe OpenAI foi a mais penalizada, com um Recall de apenas 14%. A rede demonstrou severas dificuldades em isolar a "assinatura" deste modelo específico, sugerindo que os textos produzidos pelo ChatGPT são estruturalmente mais "genéricos" ou que sofrem uma sobreposição de estilo direta com os restantes LLMs (como o Claude e o Gemini).

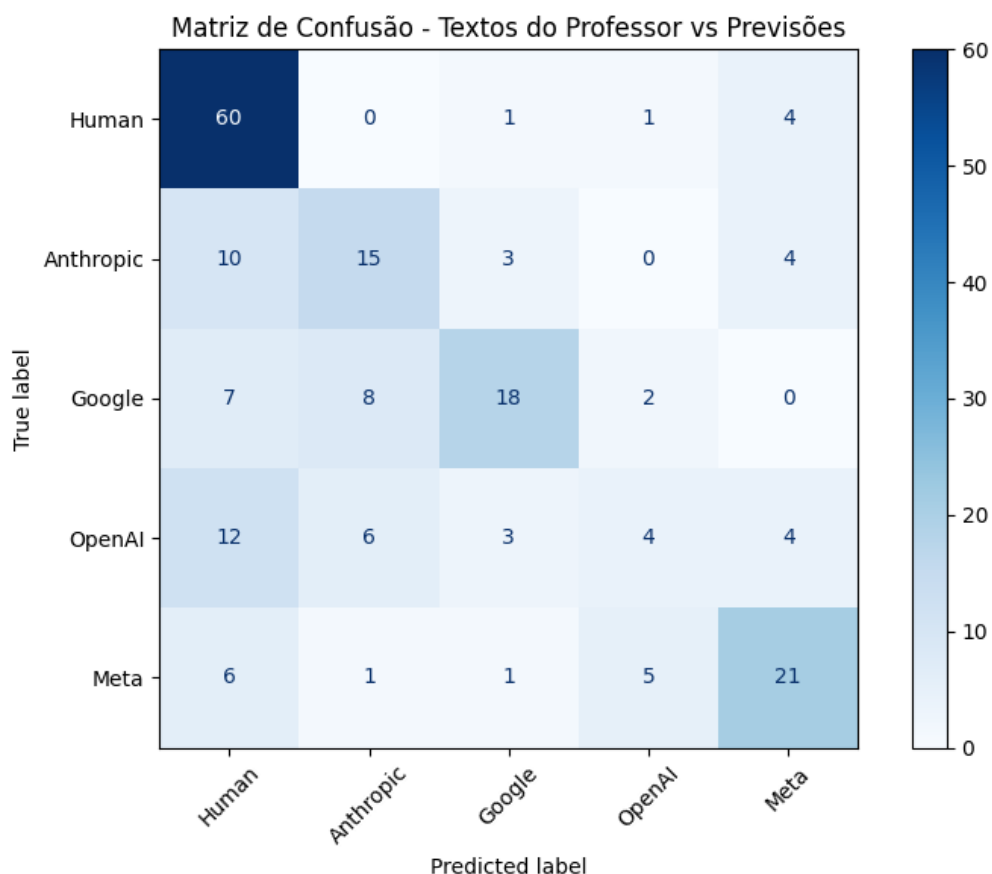


Figure 4: Matriz de confusão

6 Conclusão

Ao longo deste projeto, foram exploradas diferentes abordagens para a classificação de textos, começando por um modelo desenvolvido de raiz em Numpy e evoluindo para soluções mais avançadas com recurso ao PyTorch.

A implementação de raiz permitiu compreender em detalhe os fundamentos do treino de redes neurais, incluindo propagação direta e inversa, funções de ativação, funções de perda e técnicas de otimização como momentum, regularização L2 e early stopping. Apesar da sua simplicidade, este modelo demonstrou um desempenho elevado, especialmente quando combinado com técnicas eficazes de representação textual como TF-IDF e bi-grams.

Posteriormente, a utilização do PyTorch possibilitou a experiencição com arquiteturas mais complexas, como modelos baseados em embeddings e mecanismos de atenção, resultando numa melhoria significativa do desempenho e da capacidade de generalização.

De forma global, os resultados obtidos evidenciam a importância da engenharia de features e da escolha adequada do modelo, mostrando que soluções simples podem ser bastante eficazes.

Em conclusão, o projeto permitiu consolidar conhecimentos fundamentais de machine learning e deep learning, bem como demonstrar a evolução natural de modelos desenvolvidos manualmente para frameworks mais robustas e escaláveis.